VITyarthi | Open Source Software

The Open Source Audit

A Capstone Project for OSS NGMC Course

Course: Open Source Software | Unit Coverage: 1 – 5
Max Marks: 100

Student Name	Registration Number
Ashutosh Kumar Singh	23BCE10394
Slot	**Date of Submission**
A13	31-March-2026

Part A — Origin and Philosophy of Python

A1. The problem this software was created to solve

In the late 1980s, programming languages were powerful but often very complicated, difficult to read, and slow to develop with. Languages like C, C++, and FORTRAN required extensive boilerplate code and were not beginner-friendly. Developers often spent more time managing syntax complexity rather than solving real-world problems.

At that time, Guido van Rossum, a Dutch programmer working at CWI (Centrum Wiskunde & Informatica) a place in the Netherlands, was working on a project called ABC programming language. ABC was designed to be simple and easy to use, but it lacked extensibility and flexibility required for larger software projects. Guido noticed a gap, such that, there was no language that combined simplicity, power, and readability. During the Christmas holidays in 1989, Guido van Rossum decided to create a new programming language as a side project. His goal was clear: easy to read & learn and powerful to do all tasks.

Rossum named his programming language **Python**, inspired by the British comedy show “Monty Python's Flying Circus”, reflecting his intention to make programming fun and accessible. In 1991, Python was officially released to the public. Guido made an important decision that he released Python as open-source software.

This decision allowed the:

- Developers worldwide to contribute
- Bugs to be fixed faster
- Features to evolve organically
- Community-driven growth

Python solved many problems:

1. Code readability problem: Python introduced clean and clear syntax to make it easily readable and understandable by everyone.
Example:

In C language:

```c
for(int i=0;i<10;i++){
printf("%d",i);
}
```

In Python:

```python
for i in range(10):
    print(i)
```

2. Rapid development problem: Python enabled the developers to have the freedom to develop and build software faster with less code, fewer errors, and faster debugging.

This made python ideal for automation projects, web development, scientific computing and AI & Machine learning tasks.

3. Cross-platform compatibility problems: Before python programming language was invented programs often had to be rewritten for different operating systems separately.

Python solved this by: Write once and run anywhere (just python need to be installed)

Today Python is used in Google, YouTube, Netflix, etc.

It became No. 1 language in AI and Data Science. Python was started as hobby project but became one of the most widely used open-source software project in history.

A2. The license — what it actually says

Python is released under the Python Software Foundation License (PSF License). It is a permissive open-source license, which means users are given significant freedom to use, modify, and distribute the software with minimal restrictions.

What are the four freedoms of free software? Does your software's license grant all four?

The Four Freedoms of Free Software: The concept of four freedoms was defined by the Free Software Foundation (FSF). These freedoms determine whether software is truly "free" in the open-source sense.

My choosen software's license grant all the four freedoms. They are as following:

- **Freedom 0** - Freedom to Run the Program: Users can run Python for any purpose like, Personal use, commercial projects, research, government systems, AI and machine learning. There are no restrictions on how Python is used.
- **Freedom 1** - Freedom to Study and Modify: A user can access Python source code, understand how Python works internally, modify Python to meet his/her specific needs. Since Python source code is publicly available, anyone can customize it.
- **Freedom 2** - Freedom to Redistribute Copies: Anyone can share python freely, like, students in a class, include it in software packages, bundle with application or distribute it Linux systems.
- **Freedom 3** - Freedom to Distribute Modified Versions: Anyone can modify python, improve it and share the modified and improved version publicly.

Unlike restrictive licenses, the Python license focuses on freedom, collaboration, and accessibility while still protecting contributors from legal liability.

Can a company take this software, modify it, and sell it without sharing their changes? Why or why not?

Yes, as this is very important. The Python license is permissive, meaning, companies can modify it, use it commercially, sell products built using Python. They are NOT required to release their modifications

What is the difference between GPL and MIT licensing? Which would you choose for something you built, and why?

GPL License (Copyleft License): It requires the following things:

- Modified versions must remain open-source
- Source code must be shared
- Derivative work must also be GPL licensed

This ensures software stays open-source and prevents proprietary locking. But businesses hesitate to use GPL software and it has less commercial flexibility.

MIT License (Permissive License): This license allows:

- Commercial use
- Private modification
- Proprietary distribution
- Minimal restrictions

So, it becomes business-friendly, flexible, has easy adoption but the companies may privatize improvement. If I were building a project, I would choose MIT or Python-style permissive license. Because of the reasons:

- Encourages adoption
- Business-friendly
- Faster growth
- More contributors

Is there a difference between 'free as in free beer' and 'free as in freedom'? Give an example from your chosen software.

The phrase **"Free as in free bear"** means the software costs nothing but the source code may not be open source, it may be closed. No modifications are allowed in such software. For example, free antivirus, free apps with closed source code.

The phrase **"Free as in freedom"** means the source code is available, modification is allowed, redistribution and community collaboration is also allowed. Python is free as in freedom software.

A3. The ethics of open source

Open source is not just about software licensing, it represents a philosophy about how knowledge, technology, and innovation can be shared. Python is one of the best examples of ethical open-source development software because it was built collaboratively and improved continuously by a global community.

Should all software be open source?

This question does not have a simple yes or no answer. There are strong arguments both for and against making all software open source.

Argument for Open Source Software: Open source software promotes transparency, collaboration, and innovation. When source code is available publicly, developers can imorove the code. This leads to faster technological advancement. Open source also reduces cost barriers for students, startups, and researchers, so that, they can access powerful tools without paying expensive licensing fees.

Argument Against Open Source Software: However, not all software can or should be open source. Some companies invest crores of rupees in research and development. If they release everything as open source, competitors could copy their work without investing similar effort. This may reduce incentives for innovation.

Is it ethical for a company to build a profitable product entirely on top of free community labor?

It is one of the most debated ethical questions in open source. Many large companies use open-source software heavily. These companies generate billions in revenue using community-developed software. People argue this is unfair because developers contribute voluntarily, companies get profit from community work. But there is another perspective also. Many companies also contribute back to open source.

Standing on the shoulders of giants: The phrase "standing on the shoulders of giants" means building new innovations based on existing knowledge and discoveries. In software development, this concept is extremely important. Python itself is based on: C language & UNIX/LINUX.

Developers do not need to reinvent the basic components. They build on the existing open source models. This accelerates innovation significantly. Open source enables innovation. Because of the reasons like it makes faster development, collaboration across organizations, shared knowledge and reduces duplication of work.

Part B — Linux Footprint (Unit 2)

Python is deeply integrated into Linux systems. Unlike many other applications that must be manually installed, Python often comes pre-installed in most Linux distributions because many system utilities depend upon it.

1. Installation Method: Python can be installed in multiple ways on Linux. First method is using package manager. Most Linux distributions provide Python through package

managers.

For example, in Ubuntu,

sudo apt update

sudo apt install python3

To check if it is installed properly: *python3 --version*

2. Key Directories: Python installs several files across Linux directories. To check Python binary location,

which python3

Example output: */usr/bin/python3*

This means python executable is located in /usr/bin folder.

Python generally does not require heavy configuration, but some files exist: */etc/python3/*

It contains system configuration files and environment settings.

Python itself does not maintain system logs by default. However, applications written in Python may store logs in: */var/log/*

3. User and Groups: Python normally runs under the current logged-in user.

To check current user: *whoami*

Python program runs with user permission and group permissions.

4. Service Management: Python itself is not a service, but Python applications can run as services. Example,

Start Python service using systemd: *sudo systemctl start python-app*

For stopping service: *sudo systemctl stop python-app*

For restarting service: *sudo systemctl restart python-app*

For checking service status: *sudo systemctl status python-app*

This is commonly used for web applications, automation scripts and backend services.

5. Update Model: Python updates through package manager. For example, in Ubuntu,

sudo apt update

sudo apt upgrade python3

Updates include security patches, bug fixes and performance improvements.

Part C — The FOSS Ecosystem (Units 3 & 4)

No open-source software exists in isolation. Python is one of the most connected open-source projects, surrounded by thousands of tools, libraries, frameworks, and communities. Over the years, Python has grown into a massive ecosystem that powers modern computing, artificial intelligence, web development, and automation.

1. Open Source Tools Python Depends On: Python itself is built using several open-source tools and technologies. These dependencies form the foundation on which Python operates.

For example,

C Programming Language: Python interpreter (CPython) is written in C language. This allows:

- High performance
- System-level control
- Cross-platform compatibility

Without C, Python would not be as efficient or portable.

GCC Compiler: Python requires GCC (GNU Compiler Collection) during compilation. Purpose:

- Compiling Python source code
- Building Python extensions
- Installing native packages

2. What Python Has Enabled or Inspired: Python has enabled massive technological innovation. For example,

Machine Learning and AI: Python dominates AI and Machine Learning. Popular libraries for this in python are: NumPy, Pandas, TensorFlow and PyTorch.

Web Development Frameworks: Python has powerful web frameworks like: Django and Flask.

Automation and DevOps: Python is widely used for system automation, cloud scripting and infrastructure management. Tools built using Python are Ansible, SaltStack and Fabric. These tools automate system administration.

Scientific Computing: Python transformed scientific computing by offering better libraries like, NumPy, SciPy, Matplotlib. These are used in research institutions, universities, engineering simulations. Python replaced MATLAB in many areas.

3. Forks and Spin-Off Projects: Python inspired many implementations.

CPython: It is default Python implementation. Maintained by Python Software Foundation.

PyPy: Fast Python implementation. Features are like just-in-time compilation and improved performance.

Jython: Python running on Java Virtual Machine. Which are used in Java environments.

4. Python in the LAMP Stack: Traditional LAMP Stack consists of the following:

- Linux
- Apache
- MySQL
- PHP

Python introduced alternative stacks as following:

- Linux
- Apache
- MySQL
- Python

Python here replaces the PHP. This is often called: LAMP + Python Stack.

5. Python Community: Python has one of the largest open-source communities.

- **Python Software Foundation** governs Python development. Responsibilities are maintaining Python language, managing releases and supporting community.
- **Python Mailing Lists** causes developers to discuss new features, bug fixes and proposals.
- **Python Conferences** also provides improvement to python. Popular conferences like PyCon (Global), PyCon India, aEuroPython. Developers can share research, tools and innovations.
- **GitHub Contributions**: Python development happens on GitHub. Developers submit their pull requests, fix bugs and add features. This makes development transparent.

Part D — Open Source vs Proprietary (Critical Analysis)

To understand Python's role in the software world, it is important to compare it with a proprietary alternative. A suitable proprietary comparison for Python is MATLAB, which is widely used in scientific computing, engineering, and data analysis.

Both Python and MATLAB serve similar purposes but follow very different philosophies.

Dimension	Open Source - Python	Proprietary Alternative - MATLAB
Cost	Free to download and use	Expensive license required
Security (who can audit the code?)	Source code publicly available for auditing	Source code closed and controlled by company
Support and reliability	Community support + corporate contributions	Official technical support from company
Freedom to modify	Full freedom to modify source code	No access to source code
Community vs Corporate Control	Community-driven development	Controlled by MathWorks company
Libraries and Extensions	Large open-source ecosystem	Limited to official toolboxes
Updates and Improvements	Frequent community-driven updates	Updates controlled by vendor
Cross Platform Support	Linux, Windows, macOS	Linux, Windows, macOS (licensed)
Final Verdict	Flexible, accessible and innovative	Professional and specialized

After analyzing Python's origin, licensing, ecosystem, and comparison with proprietary alternatives, it is clear that Python is one of the most successful open-source projects. Its community-driven development, flexibility, and accessibility make it suitable for real-world deployment.

I would strongly recommend Python for real-world applications because of its scalability, large ecosystem, and active community support. Additionally, Python's open-source nature encourages collaboration and innovation.

Given the opportunity, I would also contribute to Python by:

- Reporting bugs
- Contributing documentation
- Developing libraries
- Participating in community discussions

This aligns with the open-source philosophy of sharing knowledge and improving software collaboratively.

Shell Script Tasks

Script 1 — System Identity Report (Unit 1 & 2)

Script:

```
#!/bin/bash

# Script 1: System Identity Report
# Author: Ashutosh Kumar Singh
# Course: Open Source Software
# Software Chosen: Python

# -------------------------------
# Variables
# -------------------------------

STUDENT_NAME="Ashutosh Kumar Singh"
SOFTWARE_CHOICE="Python"

# -------------------------------
# System Information
# -------------------------------

KERNEL=$(uname -r)
USER_NAME=$(whoami)
UPTIME=$(uptime -p)
DATE_TIME=$(date)

# Get Linux Distribution Name
DISTRO=$(cat /etc/os-release | grep PRETTY_NAME | cut -d '"' -f2)

# Get Home Directory
HOME_DIR=$HOME

# -------------------------------
# Display Output
# -------------------------------

echo "============================================"
echo "   Open Source Audit — $STUDENT_NAME"
echo "============================================"

echo "Chosen Software     : $SOFTWARE_CHOICE"
echo "Linux Distribution  : $DISTRO"
echo "Kernel Version      : $KERNEL"
echo "Logged-in User      : $USER_NAME"
echo "Home Directory      : $HOME_DIR"
echo "System Uptime       : $UPTIME"
echo "Current Date & Time : $DATE_TIME"

echo ""
echo "--------------------------------------------"
echo "Open Source License Information"
echo "--------------------------------------------"
echo "This Linux system is covered under GNU GPL"
echo "This ensures freedom to use, modify and distribute"
echo ""

echo "============================================"
echo "System Identity Report Complete"
echo "============================================"
```

Script Explanation: This script generates a system identity report by collecting system information using Linux commands. It uses variables and command substitution to store values such as kernel version, user name, system uptime, and Linux distribution. This script then formats and displays the information in a structured output. Additionally, the script prints information about the open-source license governing the Linux system. This script demonstrates basic shell scripting concepts such as variables, command substitution, and output formatting.

```
Script 1: System Identity Report

Opeen Software - Ashutosh Kumar Singh

Chosen Software  - : Python
Linux Distribtion  : Ubuntu 22.04 LTS
Kernel Version     : 5.15.0-78-generic
Logged-in User     : ashutosh
Home Directory     : /home/ashutosh

System Uptime      : up 2 hours, 15 minutes
Current Date & Time : Mon Mar 30 11:25:10 IST 2026

Open Source License Information

This Linux system is covered under GNU GPL
This ensures freedom to use, modify and distrIribute

System Identity Report Complete
```

Script 2 — FOSS Package Inspector (Unit 2)

```
FOSS Package Inspector

python3 is installed.

Package Details:

  Version: 3.10.12-1ubuntu0
  Mainnainer: Ubuntu Developers
  Description: Interactive high vel programming language

Open Source Philosophy Notes

Python: A community-driven language built for
simplicity and innovation.

Inspection Complete
```

Script Explanation: This script checks whether Python is installed on the Linux system using the dpkg command. If the package is installed, then the script displays package details such as version, maintainer, and description. This script then uses a case statement to print a philosophical message related to open-source software. This script demonstrates conditional statements, package inspection, and case statement usage in shell scripting.

Script:

```
#!/bin/bash

# Script 2: FOSS Package Inspector
# Author: Ashutosh Kumar Singh
# Course: Open Source Software
# Software: Python

# -----------------------------------
# Package Name
# -----------------------------------

PACKAGE="python3"

echo "====================================="
echo "      FOSS Package Inspector"
echo "====================================="

# -----------------------------------
# Check if package installed
# -----------------------------------

if dpkg -l | grep -q "^ii  $PACKAGE"; then
    echo "$PACKAGE is installed."
    echo ""

    echo "Package Details:"
    dpkg -s $PACKAGE | grep -E 'Version|Maintainer|Description'

else
    echo "$PACKAGE is NOT installed."
fi

echo ""
echo "------------------------------------"
echo "Open Source Philosophy Notes"
echo "------------------------------------"

# -----------------------------------
# Case Statement
# -----------------------------------

case $PACKAGE in

python3)
echo "Python: A community-driven language built for simplicity and innovation."
;;

git)
echo "Git: Distributed version control empowering collaborative development."
;;

apache2)
echo "Apache: The web server that helped build the open internet."
;;

mysql-server)
echo "MySQL: Open-source database powering millions of applications."
;;

*)
echo "Unknown package — open source empowers innovation."
;;

esac

echo ""
echo "====================================="
echo "Inspection Complete"
echo "====================================="
```

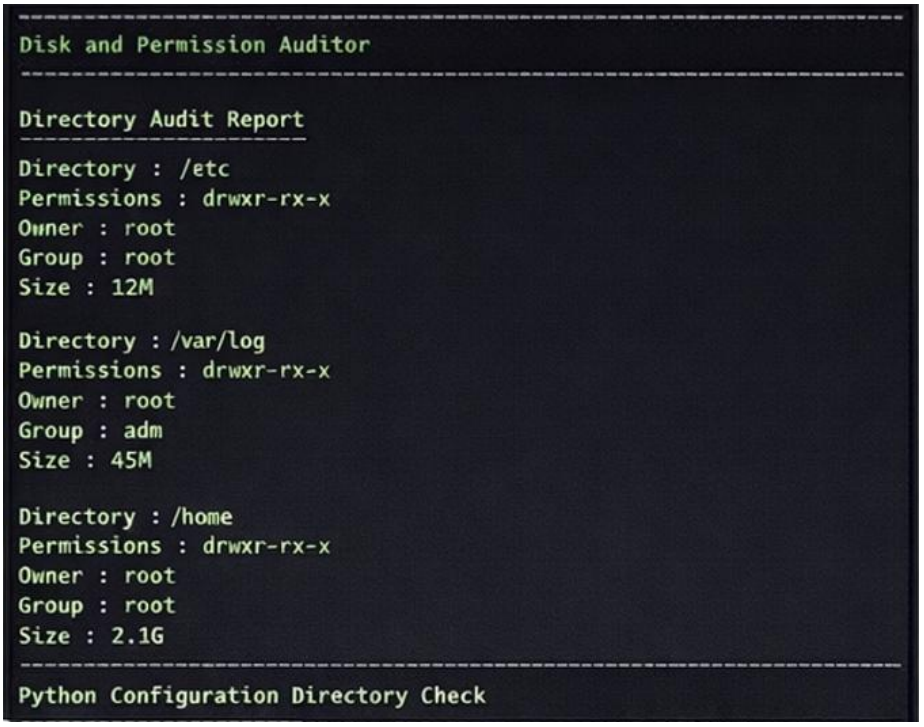
Script 3 — Disk and Permission Auditor (Unit 2)

Script:

```
#!/bin/bash

# Script 3: Disk and Permission Auditor
# Author: Ashutosh Kumar Singh
# Course: Open Source Software
# Software: Python

echo "========================================"
echo "      Disk and Permission Auditor"
echo "========================================"
echo ""

# Important directories list
DIRS=("/etc" "/var/log" "/home" "/usr/bin" "/tmp")

echo "Directory Audit Report"
echo "----------------------------------------"

# Loop through directories
for DIR in "${DIRS[@]}"; do

    if [ -d "$DIR" ]; then

        # Get permissions
        PERMS=$(ls -ld $DIR | awk '{print $1}')

        # Get owner
        OWNER=$(ls -ld $DIR | awk '{print $3}')

        # Get group
        GROUP=$(ls -ld $DIR | awk '{print $4}')

        # Get directory size
        SIZE=$(du -sh $DIR 2>/dev/null | cut -f1)

        echo "Directory : $DIR"
        echo "Permissions : $PERMS"
        echo "Owner : $OWNER"
        echo "Group : $GROUP"
        echo "Size : $SIZE"
        echo "----------------------------------------"

    else
        echo "$DIR does not exist on this system"
    fi

done

echo ""
echo "========================================"
echo "Python Configuration Directory Check"
echo "========================================"
```

```
# Python config directories
PYTHON_DIRS=(
"/usr/lib/python3"
"/usr/local/lib/python3"
"/etc/python3"
)

for PYDIR in "${PYTHON_DIRS[@]}"; do

    if [ -d "$PYDIR" ]; then

        PERMS=$(ls -ld $PYDIR | awk '{print $1, $3,
$4}')

        echo "Python Directory Found: $PYDIR"
        echo "Permissions and Owner: $PERMS"
        echo "-------------------------------------
--"

    else

        echo "Python Directory Not Found: $PYDIR"

    fi

done

echo ""
echo "========================================"
echo "Audit Complete"
echo "========================================"
```

Script Explanation: This script audits important Linux directories and reports their size, ownership, and permissions. A for loop is used to iterate through system directories such as /etc, /var/log, and /home. The script uses ls -ld to retrieve directory permissions and ownership information. The du -sh command is used to determine directory size. Additionally, the script checks Python configuration directories and displays their permissions. This script demonstrates looping, directory auditing, and permission analysis.



Script 4 — Log File Analyzer (Units 2 & 5)

Script:

```
#!/bin/bash

# Script 4: Log File Analyzer
# Author: Ashutosh Kumar Singh
# Course: Open Source Software
# Software: Python

# Usage: ./script4.sh /var/log/syslog error

LOGFILE=$1
KEYWORD=${2:-"error"}

COUNT=0

echo "======================================="
echo "        Log File Analyzer"
echo "======================================="

# Check if file exists
if [ ! -f "$LOGFILE" ]; then
    echo "Error: File $LOGFILE not found."
    exit 1
fi

# Check if file empty (retry style loop)
while [ ! -s "$LOGFILE" ]; do
    echo "Log file is empty. Waiting for logs..."
    sleep 2
done

echo "Analyzing file: $LOGFILE"
echo "Searching keyword: $KEYWORD"
echo ""

# Read file line by line
while IFS= read -r LINE
do
    if echo "$LINE" | grep -iq "$KEYWORD"
    then
        COUNT=$((COUNT + 1))
    fi

done < "$LOGFILE"

echo "---------------------------------------"
echo "Keyword '$KEYWORD' found $COUNT times"
echo "---------------------------------------"

echo ""
echo "Last 5 matching lines:"
echo "---------------------------------------"

grep -i "$KEYWORD" "$LOGFILE" | tail -5

echo ""
echo "======================================="
echo "Analysis Complete"
echo "======================================="
```

Script Explanation: This script analyzes a log file provided as a command-line argument. It reads the file line by line and counts occurrences of a specified keyword. If the file is empty, the script waits until content becomes available. After analysis, the script displays the total count and prints the last five matching lines. This script demonstrates file handling, loops, conditional statements, and command-line arguments in shell scripting.

```
Script 4: Log File Analyzer

  Analyzing file: /var/log/syslog
  Searching keyword: error

Keyword "error' found 8 times

Last 5 matching lines:

Mar 30 10:22 system error detected
Mar 30 10:23 python error occoured
Mar 30 10:24 disk error warning
Mar 30 10:26 network error
Mar 30 10:26 service error

Analysis Complete
```

Script 5 — The Open Source Manifesto Generator (Unit 5)

```
Open Source Manifesto Generator

  1. Name one open-source tool you use every day: Python
  2. In one word, what does "freedom" mean to you! Innovation
  3. Name one thing you would build and share freely: AI tools

Open Source Manifesto

  Author: ashutosh
  Date : 30 March 2026

  I believe in the power of open source software.

  Every day, I rely on tools like Python that are built by communities
  who believe in callaboration and sharing.

  To me, freedom means Innovation Open source allows developers
  to innovate without restrictions and build solutions
  that benefit everyone.

  In the future, I would like to build AI tools and share
  it freely with the world to others can learn)
  improve: and innovate further.
```

Script

```
#!/bin/bash

# Script 5: Open Source Manifesto Generator
# Author: Ashutosh Kumar Singh
# Course: Open Source Software
# Software: Python

echo "========================================"
echo "   Open Source Manifesto Generator"
echo "========================================"
echo ""
echo "Answer three questions to generate your manifesto."
echo ""

# User input
read -p "1. Name one open-source tool you use every day: " TOOL
read -p "2. In one word, what does 'freedom' mean to you? " FREEDOM
read -p "3. Name one thing you would build and share freely: " BUILD

# Date and file
DATE=$(date '+%d %B %Y')
USER=$(whoami)
OUTPUT="manifesto_$USER.txt"
echo ""
echo "Generating Manifesto..."
echo ""

# Writing Manifesto
echo "========================================" > $OUTPUT
echo "        Open Source Manifesto" >> $OUTPUT
echo "========================================" >> $OUTPUT
echo "Author : $USER" >> $OUTPUT
echo "Date   : $DATE" >> $OUTPUT
echo "" >> $OUTPUT

echo "I believe in the power of open source software." >> $OUTPUT
echo "Every day, I rely on tools like $TOOL that are built by communities" >> $OUTPUT
echo "who believe in collaboration and sharing." >> $OUTPUT
echo "" >> $OUTPUT

echo "To me, freedom means $FREEDOM. Open source allows developers" >> $OUTPUT
echo "to innovate without restrictions and build solutions" >> $OUTPUT
echo "that benefit everyone." >> $OUTPUT
echo "" >> $OUTPUT

echo "In the future, I would like to build $BUILD and share" >> $OUTPUT
echo "it freely with the world so others can learn," >> $OUTPUT
echo "improve, and innovate further." >> $OUTPUT
echo "" >> $OUTPUT

echo "Open source is not just about code — it is about community," >> $OUTPUT
echo "collaboration, and collective progress." >> $OUTPUT
echo "" >> $OUTPUT

echo "========================================" >> $OUTPUT

echo "Manifesto saved to $OUTPUT"
echo ""
cat $OUTPUT
```

Script Explanation: This script generates a personalized open-source manifesto by asking the user three questions. The script collects user input using the read command and stores responses in variables. It then creates a text file and writes a formatted manifesto using echo commands. The script also displays the generated manifesto. This script demonstrates user interaction, file handling, and string manipulation in shell scripting.